สรุปผลการเปรียบเทียบ Airflow + PySpark vs Airflow + Pandas

จากการทดสอบและจับเวลาได้ไฟล์ที่สามารถสรุปเป็นตารางได้ดังนี้:

Dataset Size	Engine	Extract (s)	Transform (s)	Load (s)	Total (s)
10k	Pandas	0.85	0.22	1.1	2.17
	PySpark	0.75	20.37	32.19	53.31
100k	Pandas	2.36	1.36	9.8	13.51
	PySpark	2.4	24.17	108.39	134.97
1m	Pandas	19.04	9.53	102.11	130.68
	PySpark	19.36	38.97	955.37	1,013.70

จากตัวเลขข้างต้น เมื่อข้อมูลโตจาก 10k เป็น 1m แถว เวลาในการประมวลผลแต่ละเฟสจึงมีการเปลี่ยนแปลง เพื่อคำนวณว่าเวลาเพิ่มขึ้นหรือลดลงกี่เท่าจึงนำไปสู่ตารางด้านล่างเพื่อคำนวณอัตราการขยายตัวของแต่ละเฟส

ตารางวิเคราะห์เปรียบเทียบประสิทธิภาพและอัตราการขยายตัว

Phase	Dataset	PySpark (s)	Pandas (s)	Faster	Speedup	Ratio
Extract	10k	0.7460s	0.8470s	PySpark	PySpark เร็วกว่า 12%	1.1x
Extract	100k	2.405s	2.359s	Pandas	Pandas เร็วกว่า 2%	1.0x
Extract	1m	19.362s	19.041s	Pandas	Pandas เร็วกว่า 2%	1.0x
Transform	10k	20.374s	0.2230s	Pandas	Pandas เร็วกว่า 99%	91.4x
Transform	100k	24.173s	1.356s	Pandas	Pandas เร็วกว่า 94%	17.8x
Transform	1m	38.967s	9.534s	Pandas	Pandas เร็วกว่า 76%	4.1x
Load	10k	32.186s	1.104s	Pandas	Pandas เร็วกว่า 97%	29.2x
Load	100k	108s	9.798s	Pandas	Pandas เร็วกว่า 91%	11.1x
Load	1m	955s	102s	Pandas	Pandas เร็วกว่า 89%	9.4x

จากตารางจะเห็นตัวเลขที่ผ่านการคำนวณมาแล้วแต่ความแตกต่างนี้เห็นได้ชัดยิ่งขึ้นในกราฟด้านล่าง ดังต่อไปนี้

การเปรียบเทียบเวลาประมวลผล ETL แต่ละ Phase ระหว่าง Airflow+PySpark และ Airflow+Pandas

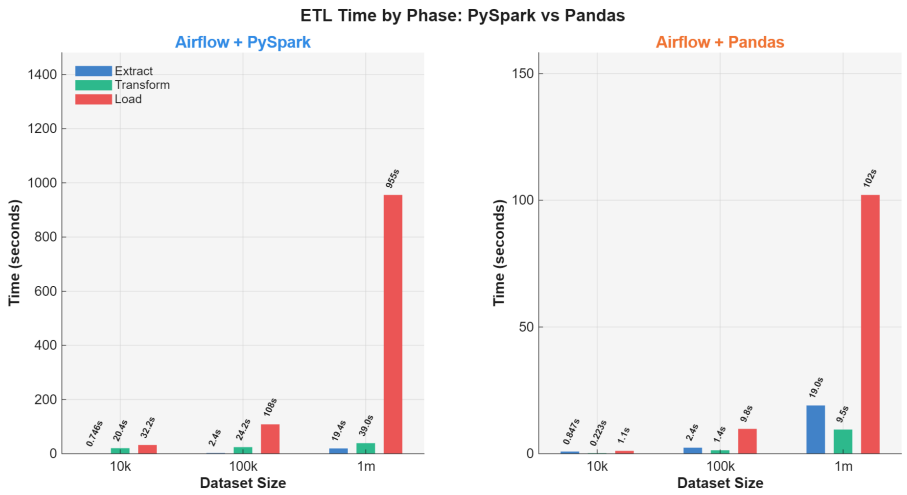

Fig 1 : การเปรียบเทียบเวลาประมวลผล ETL แต่ละ Phase ระหว่าง Airflow+PySpark และ Airflow+Pandas

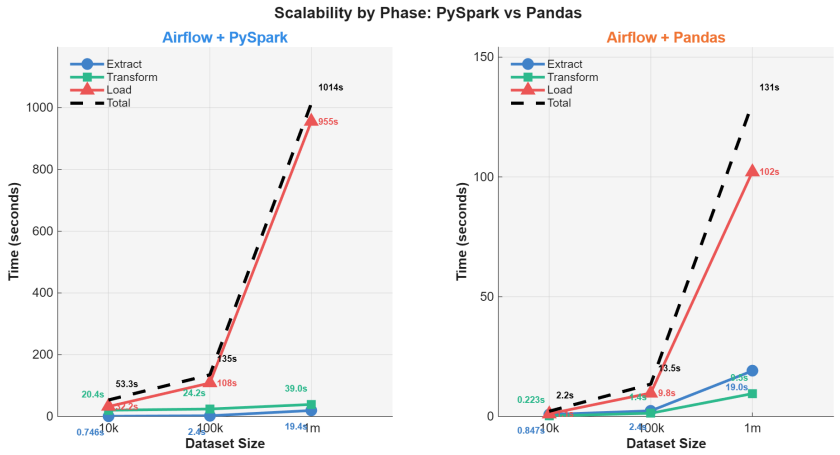

Fig 2 : อัตราการขยายตัวของแต่ละ Phase ระหว่าง Airflow+PySpark และ Airflow+Pandas

จากกราฟจะขอแบ่งการอธิบายเป็นแต่ละเฟส ดังนี้

Extract :

ผลที่ได้คือ PySpark ใช้ 0.746s / 2.405s / 19.362s และ Pandas ใช้ 0.847s / 2.359s / 19.041s ต่างกันไม่เกิน 0.3 วินาที เหตุผลมาจากโค้ดของทั้งสอง DAG ใช้ `pyodbc` เป็น Connector เดียวกันในการดึงข้อมูลจาก SQL Server โดย PySpark ใช้ `cursor.fetchall()` และ Pandas ใช้ `pd.read_sql()` ซึ่งภายในก็เรียก pyodbc เช่นกัน เมื่ออ่านข้อมูลผ่าน pyodbc ตัว Driver จะจัดการ Buffer และ Format ผลลัพธ์เป็น Python Object โดยความแตกต่างด้านประสิทธิภาพระหว่างวิธีการ Fetch ต่างๆ นั้นแทบไม่มีนัยสำคัญ**[8]** ดังนั้นเวลาที่ใช้จึงถูกกำหนดโดยปริมาณข้อมูลที่ต้องรับจาก SQL Server เป็นหลัก ไม่ใช่ตัว Engine ที่ใช้ประมวลผล

Transform :

PySpark เป็นเครื่องมือที่ให้ Python ไปควบคุม Apache Spark ซึ่งเป็นระบบที่ทำงานบน JVM (Java Virtual Machine) โดยมีชั้นกลางที่ชื่อว่า Py4J ทำหน้าที่เป็นสะพานรับส่งคำสั่งระหว่าง Python กับ JVM DEV Community พูดง่ายๆ คือทุกครั้งที่รัน PySpark มันต้องเปิดระบบ Java ขึ้นมาก่อนเสมอ**[9]** ใน Transform Phase
ผลที่ได้คือ PySpark ใช้ 20.374s / 24.173s / 38.967s ส่วน Pandas ใช้ 0.223s / 1.356s / 9.534s ต่างกันสูงสุดถึง 91 เท่าที่ Dataset 10k สาเหตุหลักคือ ทุกครั้งที่รัน PySpark มันจะเริ่มต้นเป็น Python Process ก่อน แล้วค่อยเปิด JVM Instance ขึ้นมา ทำให้แม้ Dataset จะมีแค่ 10,000 แถว เวลา Transform ก็ยังสูงถึง 20 วินาที

Load :

ผลที่ได้คือ PySpark ใช้ 32.186s / 108.389s / 955.372s ส่วน Pandas ใช้ 1.104s / 9.798s / 102.108s Load Phase นี้คือจุดที่มองเห็นได้ชัดในกราฟ สาเหตุที่ Pandas เร็วกว่าเพราะส่งข้อมูลหลายรายการพร้อมกันในครั้งเดียว ทำให้ใช้เวลารวมน้อยกว่ามาก**[10]** ส่วน PySpark ช้ากว่านั้นยังไม่สามารถระบุได้อย่างชัดเจนมากนัก แต่จากการวิเคราะห์คิดว่าเป็นเพราะวิธี

ที่มันส่งข้อมูลเข้าฐานข้อมูลนั้นมาจาก JDBC ที่ไม่ได้ตั้งค่าแบบส่งข้อมูลหลายรายการพร้อมกันในครั้งเดียวซึ่งส่งผลให้มีการส่งข้อมูลทีละแถว เมื่อข้อมูลมี 1,000,000 แถว PySpark จะส่งข้อมูลทีละแถวและทุกรอบต้องรอให้ MySQL ตอบรับก่อนจึงจะส่งรอบถัดไปได้ ทำให้เวลารวมพุ่งสูงถึง 955 วินาที

References

[8]“pyodbc Cursor object,” *Google.com*. [Online]. Available: https://groups.google.com/g/pyodbc/c/7gTAU3MIxto?pli=1

[9]B. Steadman, “Python, spark and the JVM: An overview of the PySpark runtime architecture,” *DEV Community*, 03-May-2020. [Online]. Available: https://dev.to/steadbytes/python-spark-and-the-jvm-an-overview-of-the-pyspark-runtime-architecture-21gg

[10]*Mysql.com*. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlcursor-executemany.html